KiCad AI Production & Quoting Assistant — Specification

Oct 28, 2026 · @R. Morgan

1. Overview

KiProd (working name) is a Python application that turns a KiCad 10+ project into a complete, decision-ready production package: a clean BOM, design review findings, supplier-formatted BOMs, quotes for parts, bare boards and assembly, and a ranked comparison. Engineers keep designing; the agent chases quotes and writes the paperwork.

The same core engine runs four ways: a KiCad 10 IPC plugin, a legacy SWIG action-plugin fallback, a standalone desktop GUI, and a CLI run from any KiCad project folder.

Goals

- Produce a normalized BOM from the schematic, even when the project has none.
- Resolve under-specified parts using user-defined defaults, prompts, error logs or AI inference.
- Review schematics, PCB and docs, and report production risks before money is spent.
- Quote every enabled supplier at every requested quantity, through API, web or signed email RFQ.
- Emit a report and spreadsheet per supplier, plus a cross-supplier comparison.
- Leave every artifact in predictable, user-configurable folders under the project.
- Treat suppliers and producers as drop-in plugins, so new services need no core changes.

Non-goals (v1)

- Placing purchase orders or paying automatically. Ordering stays a human action; adapters may prepare carts.
- Editing the design. The app reads the project and writes only to its own output folders, except opt-in field write-back (MPN, supplier P/N) after approval.
- Legal or customs advice. Duty figures are estimates from a user-maintained rate table.

Users

Role	Uses the app to
Design engineer	Launch a run from KiCad, fix flagged parts, read review findings
Production / purchasing	Approve RFQs, compare suppliers, choose vendors, export orders
Engineering manager	Read cost roll-ups per quantity and variant, track spend over revisions
Plugin author	Add a supplier, board house, assembler or other service
AI agent	Execute the pipeline, extract quotes from replies, draft reports

Success criteria

- A 150-line BOM is normalized, reviewed and quoted against 3 API suppliers in under 10 minutes, unattended.
- 100% of prices, stock and lead times in reports trace to a supplier response; none come from the AI.
- Every assumption the app made (default, guess, substitution) is listed in the run log and the report.
- A second run on an unchanged project reuses cached data and finishes in under 1 minute.

2. Architecture and KiCad integration

The core is a standalone Python package that never depends on KiCad's embedded Python; every KiCad entry point is a thin launcher that starts it as a separate process. This is forced by what KiCad 10 offers today.

What KiCad 10 allows (as of Oct 2026)

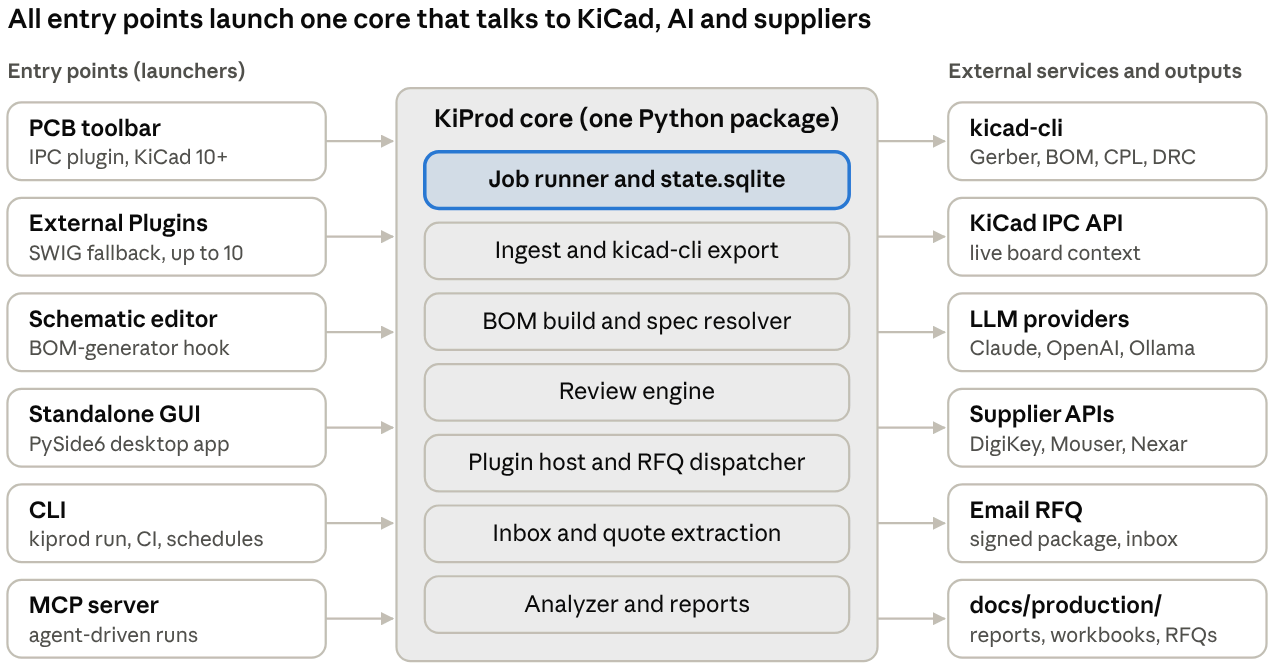
Fact	Consequence for KiProd
KiCad 10.0 shipped in March 2026 with design variants and design blocks	BOMs and quotes are computed per variant
The IPC API in KiCad 9 and 10 talks only to a running GUI and cannot plot or export	All exports go through `kicad-cli`; IPC is used only for live context (selection, board stack-up)
KiCad 9 and 10 support IPC plugins only in the PCB Editor; the Schematic Editor gains them in KiCad 11	The primary toolbar button lives in the PCB Editor; the schematic side uses the BOM-generator hook
IPC Python plugins run in a KiCad-managed virtual environment per plugin, with dependencies installed automatically	KiProd can declare heavy dependencies (PySide6, openpyxl, LLM SDKs) safely
KiCad passes `KICAD_API_SOCKET` and `KICAD_API_TOKEN` to launched plugins	The launcher forwards both to the app process
SWIG bindings are deprecated and were removed from the nightlies that become KiCad 11	The SWIG launcher is a fallback for 8.x–10.x only and contains no logic

Entry points

1. **IPC plugin (primary, KiCad 10+).** A `plugin.json` in `${KICAD_DOCUMENTS_HOME}/10.0/plugins/kiprod/` registers toolbar actions: "Run production package", "Quote only", "Open KiProd". Install through the Plugin and Content Manager.
2. **SWIG action plugin (fallback).** A `pcbnew.ActionPlugin` registered under Tools → External Plugins. It reads the board path, then spawns `python -m kiprod gui --project <dir>` with the user's configured interpreter. It holds no business logic.
3. **Schematic Editor hook.** A BOM-generator entry (Tools → Generate Legacy BOM) with the command line `kiprod run --from-netlist "%I" --project "%P"`. This gives schematic-only users a launch point until KiCad 11.
4. **Standalone GUI.** `kiprod gui` from a project folder, or a desktop shortcut that asks for a `.kicad_pro`.

5. **CLI.** `kiprod <command>` from a project folder, scriptable for CI and scheduled re-quotes (section 9).
6. **MCP server (suggested).** `kiprod mcp` exposes the same commands as tools, so Claude, ChatGPT or a local agent can drive runs conversationally.

Rule: launchers pass paths and context, never data. Everything the core needs is re-read from disk or `kicad-cli`, so a run from the CLI and a run from the toolbar produce byte-identical outputs.



KiProd architecture · 6 entry points, 1 core, 6 external services

Launchers on the left carry only paths; the core alone reads files, calls kicad-cli, and talks to AI and suppliers.

Core components

Component	Responsibility
Project loader	Finds `.kicad_pro`, `.kicad_sch` tree, `.kicad_pcb`, variants, docs; records file hashes
KiCad adapter	Wraps `kicad-cli` (netlist, BOM, pos, Gerber, drill, IPC-2581, ODB++, STEP, PDF, ERC, DRC); optional IPC client via `kicad-python`

Component	Responsibility
S-expression reader	Reads symbol fields, sheet hierarchy, board setup and stack-up directly when the CLI lacks a field
Canonical model	Pydantic models for Project, Part, BOMLine, Board, AssemblySpec, Quote, Offer
Spec resolver	Parses values, applies defaults policy, calls AI for "guess" fields
Review engine	Rule checks plus AI review; emits findings with severity and evidence
Plugin host	Loads supplier/producer plugins, validates manifests, enforces permissions
Dispatcher	Sends RFQs by API, web or email; tracks state per supplier
Inbox	Polls APIs and an IMAP mailbox; AI extracts quote data into Offer records
Analyzer	Landed cost, duties, lead time, coverage, scoring
Reporter	Markdown/HTML/PDF reports and `.xlsx` / `.csv` workbooks
Job runner	Resumable job queue in SQLite; every stage is idempotent and cached
AI layer	Provider-agnostic LLM access (section 4)

Process model

- GUI and CLI are clients of one job runner. A run survives closing KiCad or the GUI and resumes on next launch.
- State lives in `docs/production/.kiprod/state.sqlite` (path configurable): job queue, HTTP cache, RFQ tracking, audit log.
- Long waits (email replies take days) are modeled as a job in `awaiting_response`, re-checked by `kiprod inbox poll` or a scheduled task.

- GUI toolkit: PySide6, recommended for rich tables and because the IPC venv isolates it from KiCad's wxPython. A local web UI (FastAPI) is an optional second front end for headless build servers.

3. Processing pipeline

A run is a fixed sequence of idempotent stages; each writes a JSON snapshot so later stages, re-runs and the AI all work from the same facts. Stages 1–5 run without network access; stages 6–10 need it.

1. **Ingest.** Locate project files, hash them, detect KiCad file-format versions and design variants. Abort with a clear message if files are from KiCad < 8 (offer `kicad-cli sch upgrade` on a copy).
2. **Export.** Through `kicad-cli`: XML netlist, schematic PDF, ERC report, DRC report, Gerbers, drill, pick-and-place, IPC-2581 and/or ODB++, STEP, board PDF. Per variant where outputs differ. Exports go to `production/fab/` and are reused when hashes match.
3. **Build BOM.** If a BOM exists (CSV in a known location, or a KiCad BOM preset), import it and reconcile against the netlist; otherwise generate it. Group by MPN, else by value + footprint + key fields. Honor DNP, Exclude-from-BOM and variant overrides. Output `bom.normalized.json` and `bom.csv`.
4. **Resolve specs.** For each line, parse value strings (`4k7`, `100n/50V X7R`, `0R`), map footprints to packages (`R_0603_1608Metric` → 0603/1608), then apply the defaults policy (section 8) to every missing attribute: Default, Flag, Error or Guess. Every filled attribute records its source: `schematic`, `default`, `user`, `ai_guess` or `supplier`.
5. **Review.** Deterministic checks first, then AI review (below). Produce `review.md` and `review.json` with findings.
6. **Source parts.** For each enabled component supplier and each quantity in the quantity set (times board count, plus attrition): search by MPN, else parametric search; record price breaks, MOQ, packaging, stock, lead time, lifecycle, country of origin, HTS/ECCN where provided.
7. **Alternates.** For lines that are unavailable, NRND/EOL, single-source, or above a price threshold: gather candidates from supplier substitution endpoints and parametric search, then have the AI rank form-fit-function equivalence. Candidates are suggestions until a human approves them.
8. **Board and assembly RFQs.** Derive board parameters and assembly parameters (below), map them to each producer's fields, and dispatch through API, web or email.

9. **Collect responses.** API results arrive immediately; email replies are matched by RFQ number, attachments parsed (PDF, XLSX, CSV), and the AI extracts offers into the Quote schema with a confidence score. Low-confidence extractions go to a human queue.
10. **Analyze and report.** Landed cost, comparison, per-supplier reports and workbooks, executive summary (section 7).

What the review checks

Area	Deterministic checks	AI-assisted checks
BOM	Missing MPN; footprint vs package mismatch; DNP parts with MPN; duplicate refdes; parts with no footprint	Value/rating plausibility against net names (e.g. a 16 V cap on a `+24V` net); inconsistent naming of equal parts
Schematic	ERC errors and warnings; unconnected pins; power flags; unannotated symbols	Missing decoupling near ICs; missing pull-ups on open-drain buses; reversed polarity risks; datasheet-recommended parts missing
PCB	DRC results; board outline; fab rules vs selected board house capabilities; fiducials present; test points present	Assembly risks: fine pitch, bottom-side heavy parts, BGA without X-ray in spec; silkscreen polarity marks
Docs	Title block filled; revision set; fab and assembly notes present; README/CHANGELOG present	Notes contradict board setup (e.g. notes say ENIG, config says HASL)
Compliance	RoHS/REACH flags from supplier data; lifecycle status	Export-control or hazardous-material hints in descriptions

Each finding carries severity (`blocker`, `warning`, `info`), evidence (file, refdes, net, line), and a suggested fix. Blockers stop RFQ dispatch unless the user overrides them in the GUI or with `--accept-blockers`.

Board parameters extracted

Layer count; outline size and area; thickness; copper weights per layer; stack-up and dielectric; minimum track/space; minimum drill and annular ring; via types (through, blind, buried, micro, via-in-pad); surface finish; mask and silk colors; impedance-controlled nets; edge plating and castellations; slots and cutouts; gold fingers; board count per panel. Values come from board setup and stack-up first, fab notes second, defaults third.

Assembly parameters extracted

Placement count per side; unique part count; SMD vs THT vs press-fit counts; smallest package; finest pitch; BGA/QFN/LGA counts; odd-form parts; components needing programming; conformal coating keep-outs; consigned vs turnkey lines.

4. AI provider layer

The AI interprets, extracts, ranks and writes; it never invents a price, stock level, lead time or duty rate. Every AI output is a typed object validated against a JSON Schema, tagged with provider, model, prompt version and confidence.

Providers

One `LLMProvider` interface with three adapters covers every requested backend:

Adapter	Covers	Notes
`anthropic`	Claude models via the Messages API	Tool use and structured output; prompt caching for long BOM context
`openai`	OpenAI models via the Responses or Chat Completions API	Structured outputs via JSON Schema
`openai_compatible`	Ollama, llama.cpp server, vLLM, LM Studio, LocalAI, any Llama-family model served locally	Same wire format as OpenAI with a custom `base_url`; schema enforcement where the server supports it, else validate-and-retry

Adapter	Covers	Notes
`ollama` (optional native)	Ollama's native `/api/chat`	Only if a native feature (e.g. model pull, keep-alive) is needed

Models are configured by **role**, not hard-coded, so a team can mix providers:

Role	Used for	Typical choice
`reasoning`	Design review, alternate ranking, spec guessing	Strongest available cloud or large local model
`extraction`	Parsing quote emails, PDFs and spreadsheets into Offers	Mid-size model with reliable JSON output
`writer`	Report narrative, RFQ email text	Any capable model
`local_only`	Any task on projects marked confidential or export-controlled	Must be a local endpoint; cloud roles are disabled

Interface

```
class LLMProvider(Protocol):
    name: str
    def complete(self, messages: list[Message], *, schema: dict | None =
None,
                 tools: list[ToolSpec] | None = None, max_tokens: int,
                 temperature: float = 0.0) -> LLMResult: ...
    def supports(self, feature: Literal["tools", "json_schema", "vision",
"caching"]) -> bool: ...
    def estimate_cost(self, prompt_tokens: int, completion_tokens: int) ->
float | None: ...
```

Tasks are defined as Markdown prompt files with front matter (role, schema, version) in `kiprod/prompts/` and may be overridden per project or per supplier plugin.

Agent tools

For review and alternate ranking the model runs in a bounded tool loop. Tools are read-only: `get_bom_line`, `get_nets_for_ref`, `get_net_voltage_hint`, `get_footprint_geometry`, `get_datasheet_text`, `search_parts` (proxied through

enabled supplier plugins), `get_board_setup`. No tool can send email, call a write API or edit files.

Guardrails

- **Deterministic first.** Regex/unit parsing, footprint maps and rule checks run before any model call; the AI only fills what rules cannot.
- **Provenance.** Any field filled by AI is marked `ai_guess` with a rationale and shown highlighted in the GUI and reports.
- **Grounding.** Quote extraction must cite the source span (page/cell) for each number; numbers without a span are rejected.
- **Data minimization.** Cloud models receive BOM text, net names and board metrics, never Gerbers, STEP or full schematics unless the user enables it per project.
- **Budget.** Per-run token and cost caps; responses cached by content hash; a dry-run mode estimates cost first.
- **Fallback.** If no model is configured, every "Guess" policy degrades to "Flag" and the pipeline still runs end to end.
- **Evaluation.** A fixture set of BOMs and quote emails with known answers; `kiprod ai eval` reports extraction accuracy per configured model.

5. Supplier and producer plugins

Every supplier, board house, assembler or other service is a folder with a JSON manifest, an optional Python adapter, and Markdown docs; a supplier with no API can be added with the manifest and an email template alone.

Plugin folder

```
suppliers/
  digikey/
    manifest.json          # identity, channels, capabilities, fields,
permissions
    adapter.py             # optional: class implementing SupplierAdapter
    README.md              # human notes: account setup, quirks, contacts
    prompts/
      parse_quote.md       # optional override for AI extraction
    templates/
      bom_upload.json      # column mapping for the supplier's BOM format
      rfq_email.md         # Jinja2 email body for email RFQs
    tests/
      fixtures/            # recorded API responses / sample quote emails
```

Plugin search path, first match wins: project `docs/production/quotes/config/suppliers/` → user `~/.config/kiprod/suppliers/` → bundled. A project can pin or override any supplier.

Resource types

`components`, `pcb_fab`, `pcb_assembly`, `stencil`, `cable_harness`, `cnc`, `3d_printing`, `injection_molding`, `sheet_metal`, `laser_cutting`, `enclosure`, `test_fixture`, `programming`, `conformal_coating`, `box_build`, `logistics`. The list is open: an unknown type string is accepted and gets a generic tab.

Channels

Channel	Meaning	Example
`api`	Documented API with credentials	DigiKey Product Information v4 + Quote v4
`aggregator`	Data source covering many distributors, used for search and alternates, not for buying	Nexar/Octopart
`web_upload`	Supplier's own BOM or Gerber upload page; app prepares the files and a browser link	Mouser BOM tool, most board houses

Channel	Meaning	Example
`web_automation`	Browser automation of a web form; off by default, only where terms allow	Instant board-price forms
`email`	Signed RFQ package by email; replies parsed by the inbox	Local assemblers, machine shops
`manual`	App generates the package; a person submits it and pastes the reply back	Anything else

Manifest (abridged example)

```
{
  "$schema": "https://kiprod.dev/schemas/supplier-manifest/v1.json",
  "id": "com.example.acme-pcb",
  "name": "ACME PCB",
  "version": "1.2.0",
  "homepage": "https://acme-pcb.example",
  "region": { "country": "CN", "ships_from": ["CN"], "currency": "USD" },
  "contacts": [{ "role": "quotes", "email": "rfq@acme-pcb.example",
"pgp_key": "keys/acme.asc" }],
  "adapter": { "module": "adapter", "class": "AcmeAdapter", "min_core": "1.0"
},
  "credentials": [{ "key": "api_key", "label": "API key", "secret": true }],
  "permissions": { "network": ["api.acme-pcb.example"], "email": true,
"browser": false },
  "capabilities": {
    "pcb_fab": {
      "channels": ["api", "email"],
      "turnaround_days": { "min": 2, "standard": 5 },
      "limits": { "layers": [1, 32], "min_track_mm": 0.09, "min_drill_mm":
0.15,
                  "max_size_mm": [500, 600], "thickness_mm": [0.4, 3.2] },
      "finishes": ["HASL_LF", "ENIG", "OSP", "ImAg", "ENEPIG"],
      "certifications": ["ISO9001", "UL", "IATF16949"],
      "input_formats": ["gerber_x2", "excellon", "ipc2581", "odb++"],
      "bom_format": null
    },
    "pcb_assembly": {
```

```
      "channels": ["email"],
      "modes": ["turnkey", "consigned", "partial"],
      "bom_format": "templates/bom_upload.json",
      "cpl_format": { "columns": ["Designator", "Mid X", "Mid Y", "Layer",
"Rotation"], "units": "mm" }
    }
  },
  "fields": [
    { "key": "stackup_preset", "label": "Stack-up preset", "type": "enum",
      "options": ["JLC04161H-7628", "custom"], "default": "custom",
"applies_to": ["pcb_fab"] },
    { "key": "rush", "label": "Expedite", "type": "bool", "default": false }
  ],
  "duty": { "hts_defaults": { "pcb_fab": "8534.00.00" }, "incoterms": "DAP" }
}
```

Custom fields

`fields` uses a JSON-Schema-like subset: `string`, `text`, `int`, `float` (with `unit`), `bool`, `enum`, `multi_enum`, `date`, `file`, `secret`, `table` (nested rows), and `group`. Each field may declare `applies_to`, `visible_if`, `required_if`, `min` / `max`, `pattern` and `help` (Markdown). The GUI builds each supplier's settings panel from this list with no supplier-specific UI code. Unknown keys are preserved on save, so future plugin versions never lose data.

Adapter interface

```
class SupplierAdapter(ABC):
    def __init__(self, manifest: Manifest, settings: dict, secrets:
SecretStore, http: HttpClient): ...
    # capability discovery
    def health_check(self) -> Health: ...
    # components
    def search(self, line: BOMLine) -> list[Offer]: ...
    def price(self, mpns: list[str], qty: int) -> list[Offer]: ...
    def substitutes(self, line: BOMLine) -> list[Candidate]: ...
    # formatting
    def format_bom(self, bom: NormalizedBOM, qty: int) -> list[Path]: ...
    def format_fab_package(self, board: BoardSpec) -> list[Path]: ...
    # quoting lifecycle
    def request_quote(self, rfq: RFQ) -> RFQState: ...
    def poll(self, rfq: RFQState) -> RFQState: ...
    def parse_response(self, msg: InboundMessage) -> list[Offer] | None: ...
# None → use AI extraction
    # optional
    def prepare_cart(self, selection: Selection) -> CartLink | None: ...
```

Every method has a default: email-only suppliers inherit a generic adapter that formats the BOM from `bom_upload.json`, sends the RFQ email, and relies on AI extraction for replies.

Starter plugins to ship

Plugin	Resource	Channel	Basis
DigiKey	components	api	Product Information v4, Quote v4 and MyLists, OAuth 2.0; Quote v4 accepts 300 products per call
Mouser	components	api	Separate search and cart/order API keys
Nexar / Octopart	aggregator	api	GraphQL supply data; free plan capped at 1,000 matched parts

Plugin	Resource	Channel	Basis
LCSC / JLCPCB	components, pcb_fab, pcb_assembly	web_upload, email	JLC parts library and upload formats
PCBWay	pcb_fab, pcb_assembly	api (partner), email	Partner API needs approval; assembly is quoted by a person
Generic email supplier	any	email	Template for local shops

Farnell/Newark, TME, Arrow, RS, OSH Park, Eurocircuits and Aisler are good next plugins; confirm each one's current API terms before building.

6. Quoting and RFQ dispatch

Each supplier gets one RFQ per resource type per run, covering every quantity in the quantity set, with a unique RFQ number (`KP-<project>-<rev>-<supplier>-<yyyymmdd>-<n>`) that ties requests, replies and reports together.

Quote matrix

For components, the app requests price at `board_qty × qty_per_board × (1 + attrition)` for every board quantity, then rounds up to MOQ and order multiple, and also evaluates the next price break (buying 1,000 resistors is often cheaper than 400). For boards and assembly it requests each board quantity directly, plus panel options when the producer supports them.

RFQ state machine

`draft` → `awaiting_approval` → `sent` → `acknowledged` → `quoted` | `partial` | `declined` | `expired` | `no_response`. Transitions are logged with timestamps. Reminders go out after a configurable delay (default 3 business days); `no_response` is set after a second delay (default 7 business days).

Email RFQ package

For suppliers without a usable API, the app builds a package, scans it, signs it, optionally encrypts it, and emails it after approval.

1. **Assemble.** Only what that supplier needs: supplier-format BOM, CPL/pick-and-place, Gerber + drill (or IPC-2581/ODB++), fab and assembly drawings (PDF), stack-up, assembly notes, and an `rfq.json` with quantities, required dates and requested fields. Never the KiCad source files unless the user opts in.
2. **Manifest.** `MANIFEST.sha256` lists every file with its SHA-256 hash and size.
3. **Scan.** Run ClamAV (or a configured scanner) over the package; also check for embedded paths, usernames and hidden layers. A failed scan blocks sending.
4. **Sign.** Ed25519 detached signature over the manifest (minisign format), or OpenPGP if the supplier provides a key. The public key fingerprint is printed in the email body and published at a URL the company controls. This is what makes tampering detectable.
5. **Encrypt (optional, per supplier).** To the supplier's OpenPGP or age key if they have one; else AES-256 7-Zip with a passphrase sent through a second channel (SMS, phone, or a separate email). Signing and encryption are separate settings: signing proves integrity, encryption protects confidentiality.
6. **Compress.** `.zip` by default for compatibility, `.7z` when encrypted. Packages over the size limit (default 15 MB) are uploaded to a configured file host (S3, SharePoint, Google Drive) and sent as an expiring link.
7. **Send.** SMTP with TLS through the company mail server (so SPF/DKIM pass), or Microsoft Graph / Gmail API. Subject: `RFQ KP-… — <project> rev <rev> — <resource> — qty 1/5/25/…`. Reply-to: the monitored RFQ mailbox.

The email body is generated from the supplier's `rfq_email.md` template, then reviewed by the `writer` model for clarity. It always states: company and contact, scope, quantities, required ship date, Incoterms requested, currency, response deadline, the fields the supplier is asked to return (unit and extended price per quantity, NRE/tooling, lead time, shipping cost, country of origin, HTS codes, certifications), and how to verify the package signature.

A machine-readable reply template (`quote_response.xlsx`) is attached. Suppliers that fill it in get parsed with no AI at all.

Inbox processing

- IMAP (or Graph/Gmail API) poll of the RFQ mailbox; messages matched by RFQ number in subject or headers.
- Attachments converted to text (PDF via text extraction, then OCR when needed; XLSX/CSV directly).

- Supplier `parse_response` first; AI `extraction` role second; each Offer stores the source file and page/cell for every number.
- Offers with confidence below 0.85, or with numbers that disagree with the request (wrong quantity, missing lines), go to the Quotes Inbox tab for a person to confirm.
- Questions from suppliers ("Is ENIG required?") are surfaced as tasks; the app drafts a reply but does not send it without approval.

Board and assembly specifics

- Board quotes are requested for each enabled board house, with the extracted board parameters translated to that house's field names via the manifest. Out-of-capability parameters (e.g. 0.075 mm track at a house whose limit is 0.09 mm) are flagged before sending, not after.
- Assembly quotes can be turnkey (assembler buys parts), consigned (you ship parts), or partial. In turnkey mode the assembler's BOM coverage is itself a comparison column: parts they cannot source are listed with the cheapest external source.
- Combined fab + assembly houses are quoted both as a bundle and as separate fab and assembly lines, so a bundle can be compared against "board from A, assembly at B".

7. Analysis and reports

The comparison ranks suppliers by landed cost per finished board at each quantity, and shows coverage, lead time, quality and risk beside it, so the cheapest option is never chosen blind.

Landed cost

For each supplier and quantity:

```
\text{Landed} = \sum_{lines}\left(p_{\text{break}} \cdot
q_{\text{buy}}\right) + \text{NRE} + \text{shipping} + \text{duty} +
\text{fees} + \text{FX margin} + \text{payment fees}
```

where q_buy is the required quantity rounded to MOQ and order multiple, and duty = customs value × the sum of applicable rates for each HTS code and country of origin. Fees include broker and merchandise processing fees. Per-board cost = landed ÷ board quantity.

Duties and import fees

Duty rates are data, not code: a user-maintained `duty_rates.toml` holds HTS code × origin × destination → list of rate layers, each with source URL and as-of date. Reports print the as-of date next to every duty figure, and the app warns when a rate is older than 30 days.

This matters because US rates on China-origin PCBs changed repeatedly in 2026. As one example, a CBP ruling shows bare PCBs under HTS 8534.00.0040 at a free base rate plus a 25% Section 301 duty. Industry reporting from August 2026 described a combined rate near 37.5% for non-excluded boards, with exclusions for some 2- and 4-layer boards due to expire November 9, 2026. Duty-free de minimis treatment is unavailable for China and Hong Kong origin goods. The spec makes no claim about today's rate; the app reads whatever the rate table says and shows its date.

Duty estimates assume the HTS code supplied by the supplier, else the plugin's default (e.g. 8534 for bare boards). Assembled boards often classify differently from bare boards, so PCBA lines default to "HTS needed" and are flagged rather than guessed.

Comparison dimensions

Dimension	How it is measured
Cost	Landed cost per board at each quantity; price-break savings available
Coverage	% of BOM lines quoted; list of lines not offered, with best alternative source
Turnaround	Supplier lead time + shipping transit + customs allowance; earliest ship date
Stock risk	Lines where stock < 2× need; single-source lines; NRND/EOL lines
Quality	Authorized-distributor status; certifications (ISO 9001, IATF 16949, AS9100, ISO 13485, UL); IPC class offered; inspection (AOI, X-ray, e-test); internal history (on-time %, defect rate)
Compliance	RoHS/REACH data, country of origin, export classification

Dimension	How it is measured
Terms	Currency, Incoterms, payment terms, quote validity, NRE/tooling

An overall score is a weighted sum with user-set weights (default: cost 40, turnaround 20, quality 20, coverage 15, risk 5). The score never hides the raw numbers; it only orders the table.

Outputs per supplier

- `report.md` and rendered `report.pdf`: summary, quoted lines, gaps, alternates suggested, assumptions, risks, questions for the supplier.
- `quote.xlsx` with sheets: **Summary** (per-quantity totals), **Lines** (one row per BOM line × quantity: MPN, supplier P/N, catalog number, price break used, ext. price, stock, lead time, lifecycle, origin, HTS), **Not offered**, **Alternates**, **Landed cost**, **Raw** (unaltered response data).
- Supplier-format files exactly as the supplier wants them (`bom_<supplier>.csv`, `cpl_<supplier>.csv`, fab zip).
- `raw/`: every API response and email, unmodified, for audit.

Comparison outputs

- `comparison.md` / `comparison.pdf`: a one-page recommendation per quantity, then detail.
- `comparison.xlsx`: **Matrix** (supplier × quantity landed cost), **Best mix** (cheapest source per line, with split-order penalty for extra shipping), **Coverage gaps**, **Lead times**, **Duties**, **Alternates**, **Assumptions**.
- Charts: per-board cost vs quantity per supplier (log x-axis), and cost breakdown (parts, board, assembly, shipping, duty) at each quantity.

The recommendation text is written by the `writer` model from the computed tables only; it is required to cite table cells and is checked so that every number in the prose matches a number in the workbook.

8. Configuration

Settings come in four layers, each overriding the one before: built-in defaults → user/global (`~/.config/kiprod/config.toml`) → project (`docs/production/quotes/config/kiprod.toml`) → CLI flags. Secrets (API keys, SMTP passwords, signing keys) never go in these files; they live in the OS keyring, referenced by name.

The brief named the config folder `docs/qutoes/config` ; this spec reads that as `docs/production/quotes/config` . Every folder is configurable, so either layout works.

Folders

All paths are relative to the project root (the folder holding `.kicad_pro`) unless absolute. Tokens: `{project}` , `{rev}` , `{variant}` , `{date}` , `{run_id}` , `{supplier}` .

```
[paths]
production = "docs/production"
fab        = "docs/production/fab/{variant}"
quotes     = "docs/production/quotes"
config     = "docs/production/quotes/config"
runs       = "docs/production/quotes/runs/{date}_{run_id}"
state      = "docs/production/.kiprod"
existing_bom = ["bom/*.csv", "docs/*bom*.csv"]   # where to look for an
existing BOM
```

Quote quantities

```
[quantities]
boards = [1, 5, 25, 50, 100, 200, 400, 500, 1000]   # default set
attrition = { passives_0402_and_smaller = 0.10, passives = 0.05, ics = 0.02,
connectors = 0.0, min_extra = 5 }
consider_next_price_break = true
```

In the GUI the set is an editable chip list: type any positive integer and press Enter to add, click × to remove, "Reset to defaults" restores the nine values. The CLI accepts `--qty 1,10,250` (replace) or `--add-qty 2000` / `--remove-qty 400` . Duplicates are dropped and the list is kept sorted. Range shorthand `--qty 100..1000:100` is a suggested extra.

Defaults policy (components, boards, assembly)

Every attribute of every category has a **policy** and, where relevant, a **value**:

Policy	Behavior
`default`	Use the configured value; record source `default`
`flag`	Ask the user. GUI: a popup listing every part missing this attribute, with a suggested value and "apply to all similar". CLI: interactive prompt, or with `--non-interactive` treated as `error`
`error`	Do not guess; write the part to `under_constrained.csv` and the run log; the line is excluded from quotes until fixed
`guess`	AI infers the value from schematic context, nets and PCB geometry; record source `ai_guess` with rationale; falls back to `flag` if confidence < threshold
`inherit`	Use the value from the parent category or a project-wide rule
`ignore`	Attribute not needed for quoting

Policies can be scoped: global → category → footprint pattern → reference pattern (e.g. all `R1xx` on the power sheet).

Component defaults (example)

```
[components.resistor]
match = { ref_prefix = ["R"], footprint = ["Resistor_SMD:*",
"Resistor_THT:*"] }
power_w     = { policy = "default", value = 0.10 }
package     = { policy = "default", value = "0603", metric = "1608" }
value       = { policy = "flag" }
tolerance   = { policy = "default", value = "1%" }
tempco_ppm  = { policy = "default", value = 100 }
technology  = { policy = "default", value = "thick_film" }
aec_q200    = { policy = "ignore" }

[components.capacitor_ceramic]
voltage_v   = { policy = "guess", min_derating = 2.0 }   # AI reads net
voltage, applies 2x derating
dielectric  = { policy = "default", value = "X7R", if_value_below = "1nF",
else = "X5R" }
tolerance   = { policy = "default", value = "10%" }
package     = { policy = "inherit" }                        # from footprint

[components.ic]
mpn         = { policy = "error" }                          # never guess an
IC
```

Shipped categories: resistor, resistor array, ceramic/electrolytic/tantalum/film capacitor, inductor, ferrite bead, fuse/PTC, diode, TVS, LED, transistor, crystal/oscillator, connector, switch, IC, module, mechanical (standoffs, screws, heatsinks), and `other`. Users can add categories with their own match rules and attributes.

Preferences sit beside the defaults: preferred and banned manufacturers, preferred packaging (cut tape vs reel), "prefer basic parts" for JLC-style assemblers, lifecycle floor (reject NRND), minimum stock multiple, and approved vendor list.

Board defaults (example)

```
[boards.defaults]
material        = { policy = "default", value = "FR-4", tg_c = 150 }
thickness_mm    = { policy = "inherit" }          # from board setup; else
1.6
copper_outer_oz = { policy = "inherit", fallback = 1 }
copper_inner_oz = { policy = "inherit", fallback = 0.5 }
finish          = { policy = "default", value = "ENIG",
if_fine_pitch_below_mm = 0.5, else = "HASL_LF" }
solder_mask     = { policy = "default", value = "green" }
silkscreen      = { policy = "default", value = "white" }
ipc_class       = { policy = "default", value = 2 }
e_test          = { policy = "default", value = "100%" }
impedance_ctrl  = { policy = "guess" }            # AI checks net classes and
diff pairs
panelization    = { policy = "flag" }
via_treatment   = { policy = "default", value = "tented" }
```

Assembly defaults (example)

```
[assembly.defaults]
mode            = { policy = "default", value = "turnkey" }   # turnkey |
consigned | partial
sides           = { policy = "inherit" }                      # from
placement data
workmanship     = { policy = "default", value = "IPC-A-610 Class 2" }
solder          = { policy = "default", value = "lead_free" }
stencil         = { policy = "default", value = "framed" }
aoi             = { policy = "default", value = true }
xray            = { policy = "guess" }                        # true when
BGA/QFN/LGA present
conformal_coat  = { policy = "flag" }
programming     = { policy = "flag" }
functional_test = { policy = "flag" }
packaging       = { policy = "default", value = "esd_bag" }
```

Other resource types (CNC, 3D printing, molding, laser) get the same structure: a `[<type>.defaults]` table whose attributes and allowed values come from the union of enabled plugins' `fields`.

Other general options

Currency and FX source; destination country and ship-to address; Incoterms; required-by date; confidentiality level (`public` , `confidential` , `export_controlled`); AI provider per role; cost caps; approval rules (who must approve RFQs and email sends); report formats; logo and company details for RFQ letters.

9. User interface

One main window with a run bar on top, tabs in the middle, and a log/status drawer at the bottom; every supplier-type tab uses the same two-pane layout, generated from plugin manifests.

Window layout

- **Run bar:** project and revision, variant picker, quantity chips (add/remove inline), "Run all", "Quote only", "Stop", run status, cost-so-far for AI calls.
- **Tabs** (below).
- **Status drawer:** live log, job queue, warnings; filter by stage or supplier; "Open folder" for any output.

Tabs

Tab	Contents
Project	Detected files, KiCad version, variants, hashes, readiness checklist (BOM, Gerbers, CPL, notes)
BOM & Specs	Editable normalized BOM grid; source badge per cell (schematic / default / user / AI / supplier); filter "under-constrained"; write-back of MPN and supplier P/N to the schematic, after approval
Review	Findings list by severity; evidence pane (refdes, nets, file); accept / dismiss / defer with comment
Components	Supplier list (left) + selected supplier's settings and results (right)
Boards	Board houses (left) + extracted board parameters vs each house's limits, settings, results (right)

Tab	Contents
Assembly	Assemblers (left) + assembly parameters, mode (turnkey/consigned), settings, results (right)
Other services	One sub-tab per extra resource type in use: CNC, 3D printing, molding, laser, harness, enclosure
Quotes Inbox	RFQ state per supplier; inbound messages; extracted offers with source highlights; confirm / correct / reject; draft replies to supplier questions
Compare	Matrix (supplier × quantity), best mix, gaps, lead times, duties; weight sliders; charts
Reports	Generated files by run; open, regenerate, export bundle
Settings	General, Folders, Quantities, Component defaults, Board defaults, Assembly defaults, AI providers, Email & signing, Duties, Approvals, Plugins

Supplier pane (all supplier-type tabs)

- Left: list with enable checkbox, status dot (configured / missing credentials / error), channel icons (API, web, email), last quote date. Buttons: Add from catalog, Import plugin folder, Duplicate as custom.
- Right, sub-tabs: **Settings** (auto-built from manifest `fields`), **Credentials** (keyring-backed), **Capabilities** (read-only from manifest, with checks against this project), **Results** (offers for this supplier), **Docs** (renders the plugin's `README.md`).

Defaults editors

Components, Boards and Assembly defaults share one editor: category list on the left; on the right an attribute grid with columns Attribute · Policy (dropdown) · Value · Unit · Scope · Notes. A "Preview" button shows which BOM lines each change affects before saving.

Flag popup

When `flag` fields are hit, one batched dialog appears (not one popup per part): a table of parts × missing attributes, pre-filled with suggestions, with "apply to all similar", "save as project default" and "skip these lines".

CLI

```
kiprod run            [--project DIR] [--variant NAME] [--qty LIST] [--
suppliers IDS] [--no-send] [--non-interactive]
kiprod bom            build / refresh the normalized BOM only
kiprod review         run design review only
kiprod quote          query API suppliers; prepare (not send) email RFQs
kiprod rfq list|send|remind|cancel [--rfq ID]
kiprod inbox poll     fetch and parse replies
kiprod compare        recompute analysis from stored offers
kiprod report         regenerate reports and workbooks
kiprod config init|show|edit|validate|set KEY VALUE
kiprod plugins list|install PATH|test ID|new NAME
kiprod ai test|eval|cost
kiprod gui | kiprod mcp | kiprod doctor
```

Exit codes: 0 success; 2 blockers found; 3 under-constrained parts with `error` policy; 4 supplier failures; 5 configuration error. `--json` prints machine-readable results for CI. `kiprod doctor` checks Python, `kicad-cli` path and version, IPC socket, keyring, SMTP/IMAP, and each plugin's health.

10. Output folder layout

Each run writes to its own dated folder, and a `latest/` copy always points at the newest complete run, so history is kept and links stay stable.

```
<project>/
  docs/production/
    README.md                     # what is here, last run, how to re-run
    bom/
      bom.normalized.json
      bom.csv
      bom.xlsx
      under_constrained.csv
    review/
      review.md  review.pdf  review.json
      erc.rpt  drc.rpt
    fab/<variant>/
      gerber/  drill/  ipc2581/  odb/
      cpl.csv  board.pdf  assembly_top.pdf  assembly_bottom.pdf  board.step
      fab_notes.md  stackup.md
    test/
```

```
      test_plan.md                  # suggested feature (section 12)
      testpoints.csv
    quotes/
      config/
        kiprod.toml
        duty_rates.toml
        suppliers/                  # project-local plugin overrides
      latest/ -> runs/2026-10-28_r0007/
      runs/2026-10-28_r0007/
        run.json                    # inputs, hashes, versions, settings
snapshot
        run.log
        comparison.md  comparison.pdf  comparison.xlsx
        summary.md                  # one page for managers
        components/<supplier>/
          report.md  report.pdf  quote.xlsx
          bom_<supplier>.csv
          raw/                      # API responses as received
        boards/<supplier>/ ...
        assembly/<supplier>/ ...
        <resource>/<supplier>/ ...
        rfq/<rfq_id>/
          package.zip  MANIFEST.sha256  MANIFEST.sha256.minisig
          email_sent.eml
          replies/                  # inbound .eml and attachments
          offers.json
    .kiprod/                        # state.sqlite, cache; add to .gitignore
```

Naming rules: lowercase, no spaces, ISO dates, supplier IDs from the manifest's short name. Every generated file starts with a header (or a document property for XLSX/PDF) naming the run ID, project revision and KiCad file hashes, so a loose file can be traced back to its run.

Git guidance: commit `config/`, `bom/`, `review/` and `runs/*/comparison.*`; ignore `.kiprod/` and `raw/` if responses are under NDA. A `--git-tag` option tags the commit a quote run was based on.

11. Security, compliance and human control

The app may read freely, prepare anything, and query read-only APIs on its own; anything that sends design data to a new party, commits money, or speaks for the company waits for a person.

Approval gates (defaults)

Action	Default	Configurable to
Read project, export, review, query search/pricing APIs	Automatic	—
Create a quote or list in a supplier account via API	Automatic	Require approval
Send an email RFQ or upload files to a supplier	Approval per RFQ	Pre-approved supplier list
Reply to a supplier question	Approval per message	—
Write MPN / supplier P/N back to the schematic	Approval per batch	—
Accept an AI-suggested alternate part	Approval per line	—
Place an order or pay	Not supported in v1	Cart link only

Approvals are recorded in the audit log with user, time, and a hash of exactly what was approved; if the package changes after approval, approval is void.

Secrets and keys

- API keys, OAuth refresh tokens, SMTP/IMAP passwords and the signing key live in the OS keyring (Windows Credential Manager, macOS Keychain, Secret Service on Linux), never in config files or logs.
- The signing key can be per-user or per-company; the company public key fingerprint is shown in every RFQ.
- Logs redact anything matching a secret pattern; raw HTTP logging is opt-in.

Plugin trust

- Plugins are code. Each manifest declares `permissions` (network hosts, email, browser, filesystem write outside its run folder); the host enforces network allowlists through its HTTP client and refuses undeclared hosts.

- Plugins from outside the bundled set show a trust prompt on first load, with their declared permissions. Signed plugins (same Ed25519 scheme) can be pre-trusted by an admin.
- Plugin code runs in a subprocess with a restricted environment (no keyring access except its own named secrets, passed in by the host).

Data classification

Level	Cloud AI	Suppliers	Email RFQ
`public`	Allowed	All enabled	Signing required, encryption optional
`confidential` (default)	BOM text and metrics only	All enabled	Signing required, encryption recommended
`export_controlled`	Disabled; `local_only` role required	Only suppliers marked domestic and approved	Encryption required; no file-host links

The user, not the AI, sets the level. The app flags hints (e.g. parts with ECCN other than EAR99 in supplier data) and suggests raising the level.

Supplier terms and good conduct

- Prefer official APIs; respect rate limits and caching rules per manifest.
- `web_automation` is off by default, enabled per supplier only where the site's terms permit it, and never used to get around CAPTCHAs or log-ins the user has not provided.
- RFQ emails are rate-limited and always from a real, monitored mailbox, with a clear company identity.

Audit and reproducibility

`run.json` stores every input hash, KiCad and plugin versions, model names and prompt versions, settings snapshot, and every approval. Re-running a past run with `kiprod run --replay <run_id>` uses cached responses and must reproduce the same reports.

12. Suggested additional features and responsibilities

The highest-value additions are the ones that keep quotes current and connect them to what the team already owns; they are listed by priority.

Suggested features

Priority	Feature	Why it helps
P1	Revision diff	Compare BOM and quotes between two git revisions; show cost delta per change
P1	Variant cost roll-up	KiCad 10 design variants quoted side by side (e.g. "lite" vs "pro")
P1	Stock-on-hand check	Query InvenTree, Odoo or a CSV of internal stock before buying; quote only the shortfall
P1	Scheduled re-quote and watch	Weekly re-price; alert when price moves > X%, stock falls below need, or a part goes NRND/EOL
P1	Supplier scorecards	On-time delivery, quote accuracy and defect history from past runs feed the quality score
P2	Test plan generator	From test points, nets and parts: suggested bed-of-nails points, programming headers, power-up sequence, functional test checklist, and a test-fixture RFQ
P2	Production traveler	Assembly instructions, first-article inspection checklist, rework notes, labeling and packaging spec
P2	Compliance pack	RoHS/REACH declarations from supplier data, conflict-minerals (CMRT) gaps, country-of-origin summary
P2	DFM pre-check per house	Check the board against each enabled house's limits before sending; propose rule changes in KiCad's DRC syntax
P2	MCP server mode	Let Claude, ChatGPT or a local agent run and query KiProd conversationally

Priority	Feature	Why it helps
P3	ERP/PLM export	Push chosen sources to Odoo, Katana, InvenTree, Arena or a generic CSV PO
P3	Cost-down advisor	AI proposes consolidations (e.g. merge 4.7k and 4.99k), cheaper packages, basic-part substitutions, with savings per quantity
P3	Counterfeit-risk flag	Warn on broker-only, unauthorized, or far-below-market offers
P3	Kitting list	Per-assembler consigned kit list with bag labels and quantities
P3	Multi-project batching	Combine several projects' BOMs to reach better price breaks

Responsibilities

Activity	Engineer	Production / purchasing	KiProd agent	Supplier	Admin
Keep schematic fields (MPN, ratings) correct	Owns	Consulted	Flags gaps	—	—
Set component, board, assembly defaults	Owns	Consulted	Applies	—	Sets company baseline
Resolve flagged and under-constrained parts	Owns	—	Suggests	—	—
Act on review findings	Owns	Informed	Produces	—	—
Enable suppliers and keep credentials	Informed	Owns	Uses	—	Manages keys
Approve RFQ sends	Consulted	Owns	Prepares	—	—
Answer supplier questions	Consulted for technical	Owns	Drafts	Asks	—
Confirm low-confidence extracted quotes	—	Owns	Extracts	Provides	—
Maintain duty rate table	—	Owns	Warns when stale	—	—

Activity	Engineer	Production / purchasing	KiProd agent	Supplier	Admin
Choose vendors and place orders	Consulted	Owns	Recommends	Fulfills	—
Write and maintain supplier plugins	—	Requests	—	Provides API docs	Owns
Approve AI providers and data levels	—	—	Obeys	—	Owns

13. AI implementation brief

This section is written for a coding agent (Claude Code, Codex, or similar) and can be copied into the repository as `AGENTS.md`; build in the milestone order below and do not start a milestone until the previous one's acceptance tests pass.

Hard rules for the implementing agent

1. Python ≥ 3.11, typed throughout (`mypy --strict` on `kiprod/core`), Pydantic v2 models, `ruff` formatting.
2. The core package must import and run with KiCad absent; only `kiprod/kicad/` may call `kicad-cli` or `kipy`.
3. No business logic in launchers (`launchers/ipc`, `launchers/swig`, `launchers/bomgen`).
4. No price, stock, lead time or duty value may originate from an LLM. Enforce with a test that fails if an `Offer` lacks a `source_ref`.
5. All network I/O goes through `kiprod.net.HttpClient` (allowlist, retries, caching, rate limits, recording for tests).
6. Every stage is idempotent, keyed by input hash, and resumable from `state.sqlite`.
7. Nothing is sent externally (email, upload, write API) without an `Approval` record matching the payload hash.
8. Tests use recorded fixtures; no live supplier or LLM calls in CI.

Repository layout

```
kiprod/
  pyproject.toml
  AGENTS.md
  kiprod/
    cli.py                # Typer app
    core/
      models.py           # Project, Part, BOMLine, BoardSpec, AssemblySpec,
RFQ, Offer, Finding, Approval
      pipeline.py         # stage graph + job runner
      config.py           # layered TOML + schema validation
      defaults.py         # policy engine
(default/flag/error/guess/inherit/ignore)
      units.py            # value parsing: 4k7, 100n, 0R, 1%/50V/X7R
      footprints.py       # footprint → package map
    kicad/
      cli_adapter.py      # kicad-cli wrappers, version detection
      sexpr.py            # read-only .kicad_sch / .kicad_pcb parsing
      ipc.py              # optional kipy client
    review/rules/*.py
    sourcing/              # search, alternates, attrition, price-break
logic
    plugins/
      host.py  manifest_schema.json  base.py  generic_email.py
    rfq/
      package.py  sign.py  scan.py  mailer.py  inbox.py
    analysis/
      landed.py  duty.py  scoring.py
    reports/
      markdown.py  xlsx.py  pdf.py  templates/
    ai/
      provider.py  anthropic.py  openai.py  openai_compatible.py  tasks/*.md
eval/
    gui/                   # PySide6
    mcp/server.py
    net/http.py
  suppliers/               # bundled plugins: digikey, mouser, nexar,
lcsc_jlc, pcbway, generic_email
  launchers/
    ipc/plugin.json  ipc/launch.py
    swig/__init__.py       # ActionPlugin → subprocess
    bomgen/README.md       # Schematic Editor BOM-generator command line
```

```
  tests/
    projects/               # small KiCad 10 fixture projects: 2-layer, 4-
layer+BGA, variants, no-MPN
    fixtures/               # recorded API responses, sample quote
emails/PDFs
```

Milestones

#	Milestone	Acceptance tests
M1	Ingest + export + BOM (CLI)	`kiprod bom` on each fixture project produces `bom.normalized.json` matching golden files; works with no existing BOM; variants and DNP honored
M2	Config + defaults policy	Resistor with no wattage gets 0.10 W from `default`; `error` writes `under_constrained.csv` and exits 3; `flag` with `--non-interactive` behaves as `error`; quantity list add/remove/arbitrary values validated
M3	Plugin host + DigiKey, Mouser, Nexar adapters	Manifest schema validation; permission allowlist enforced; recorded-fixture quotes for every quantity; MOQ and price-break rounding correct
M4	Analysis + reports	Per-supplier `quote.xlsx` and `report.md`; `comparison.xlsx` with matrix, best mix, gaps; every number traceable to `source_ref`; duty table as-of dates printed
M5	Board + assembly parameters and RFQs	Extracted board parameters match fixture expectations (layers, size, min track/drill, finish); out-of-capability flags fire; CPL in each assembler's format
M6	Email RFQ + inbox	Package contains only allowed files; manifest hashes verify; signature verifies with public key; tampered file fails verification; ClamAV failure blocks send; inbox matches replies by RFQ number; sample PDFs extract with ≥ 95% field accuracy

#	Milestone	Acceptance tests
M7	AI layer + review	Same task runs on Anthropic, OpenAI and Ollama fixtures; schema-invalid output retried then failed cleanly; no-model configuration degrades `guess` to `flag`
M8	GUI	All tabs functional against stored run data; supplier panes generated from manifests; flag dialog batches parts
M9	KiCad launchers	IPC toolbar action launches GUI with project path in KiCad 10; SWIG fallback launches from Tools → External Plugins; BOM-generator command runs a full pipeline; outputs identical to CLI run
M10	Hardening	`kiprod doctor`, replay mode, packaging via PCM, docs

Definition of done for the project

On the 4-layer BGA fixture, `kiprod run --non-interactive --no-send` completes with three API suppliers and two email suppliers prepared, produces all reports and workbooks, lists every assumption, and a second run with no file changes finishes in under 60 seconds from cache.

Sources

- KiCad: IPC API for add-on developers (last modified 2026-09-19)
- KiCad 10.0.0 release announcement
- KiCad devlist: SWIG removed from nightlies, migrate to IPC API
- DigiKey API overview and DigiKey API changelog
- Mouser Python API (key structure)
- Octopart / Nexar API documentation and Nexar plans
- PCBWay partner API notes
- CBP ruling N336087 (HTS 8534.00.0040)
- AtlasPCB: Section 301 PCB exclusions expiring Nov 9, 2026
- Octopart Pulse: 2026 tariff reset and de minimis